

Exp 1 : Exercise in data Exploration using WEKA.

Data Exploration and Preprocessing Using WEKA

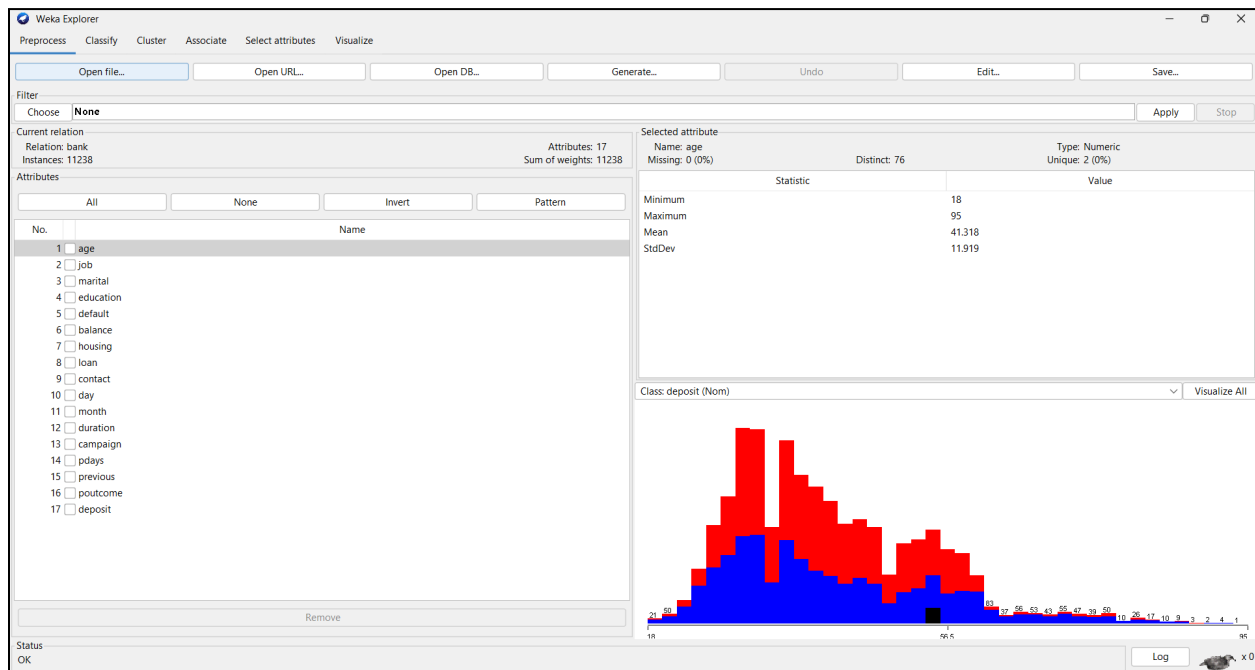
1. Loading a Real-World Dataset

Steps:

1. Open WEKA.
2. Click on **Explorer**.
3. Navigate to the **Preprocess** tab.
4. Click **Open file**.
5. Load a dataset such as [weather.arff](#), [iris.arff](#), or a **CSV** file.

Result:

- The dataset summary is displayed, including:
 - Number of instances
 - Number of attributes
 - Attribute types (numeric, nominal, date, string)



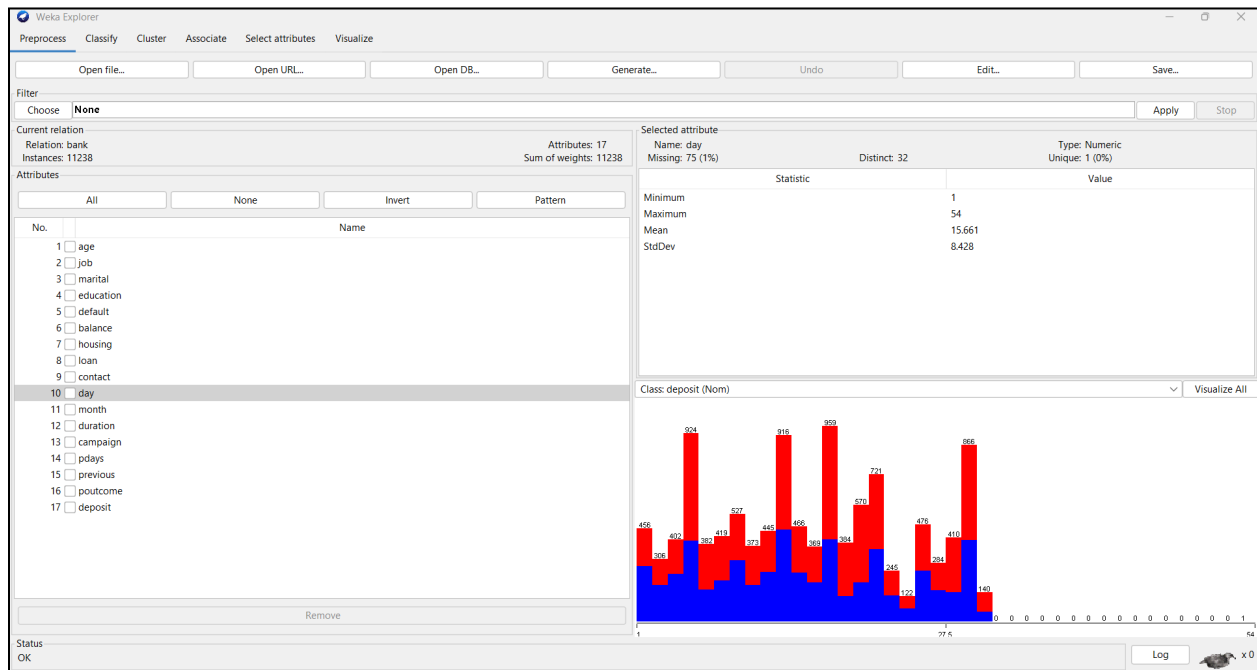
This step confirms WEKA's ability to handle real-world data formats such as CSV and ARFF.

2. Data Exploration

Attribute Analysis

- Select an attribute to view:
 - Minimum and maximum values
 - Mean and standard deviation
 - Number of distinct values

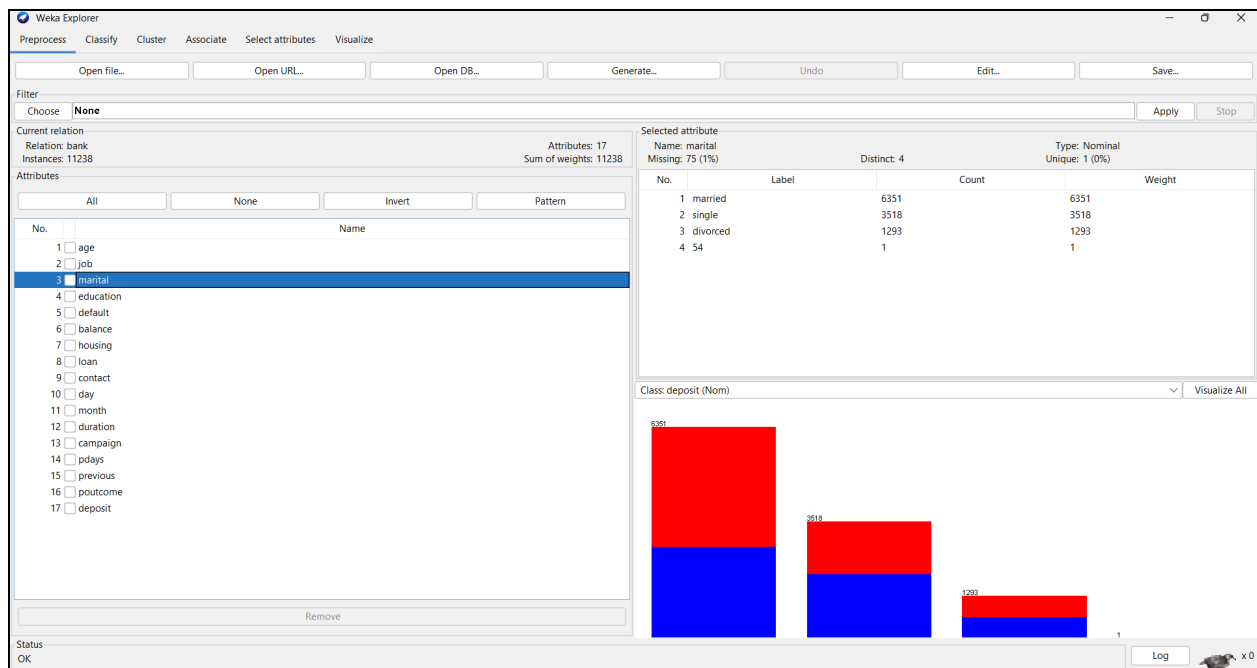
- WEKA automatically visualizes attribute distributions using histograms.



Class Distribution

- When a class attribute is set:
 - WEKA displays the distribution of instances across different classes.
 - This helps identify class imbalance.

Data exploration helps identify data quality issues before applying machine learning algorithms.



3. Handling Missing Values

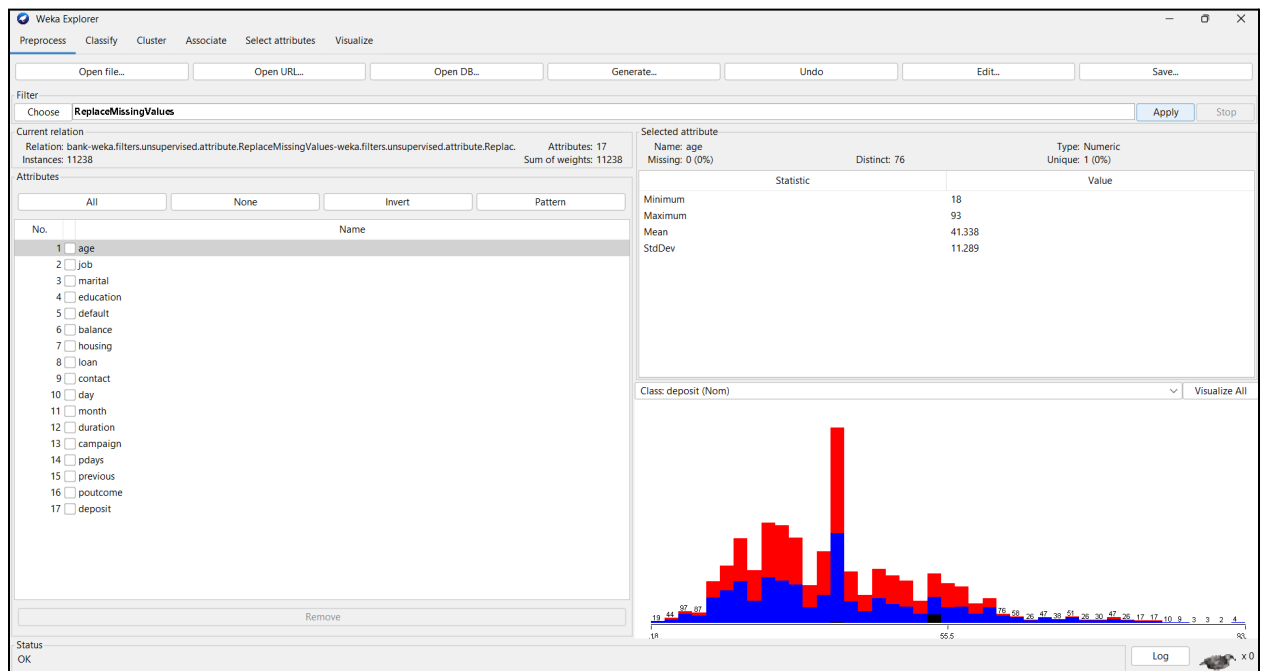
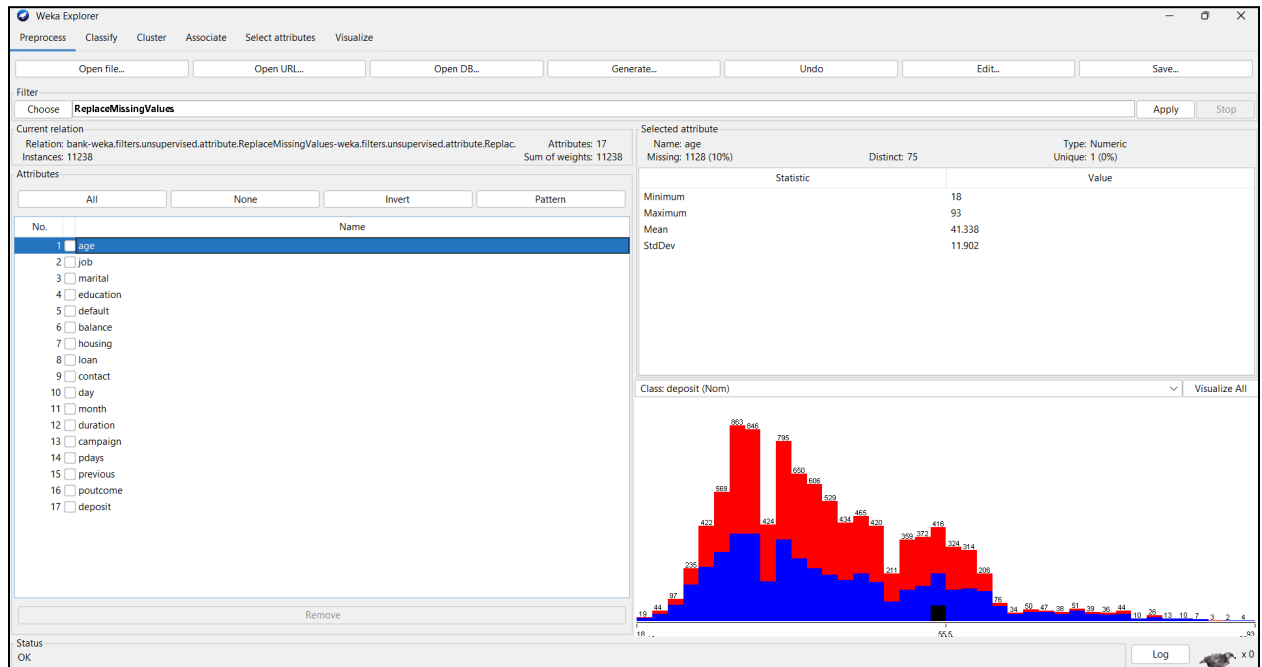
Real-world datasets often contain missing or incomplete data.

Steps:

1. Click **Filter**.
2. Select:
[Unsupervised](#) → [Attribute](#) → [ReplaceMissingValues](#).
3. Click **Apply**.

Effect:

- Numeric attributes are replaced with their mean values.
- Nominal attributes are replaced with their most frequent values (mode).



This ensures the dataset is suitable for machine learning algorithms.

4. Data Cleaning and Transformation

Removing Irrelevant Attributes

- Select unwanted attributes.
- Click **Remove**.

Attributes

All

None

Invert

Pattern

No.		Name
1	☐	age
2	☐	job
3	☐	marital
4	☐	education
5	☐	default
6	☐	balance
7	☐	housing
8	☐	loan
9	☐	contact
10	☐	day
11	☐	month
12	☐	duration
13	☐	campaign
14	☐	pdays
15	☐	previous
16	☐	poutcome
17	☐	deposit
18	☒	unnamed

Remove

Attributes

All

None

Invert

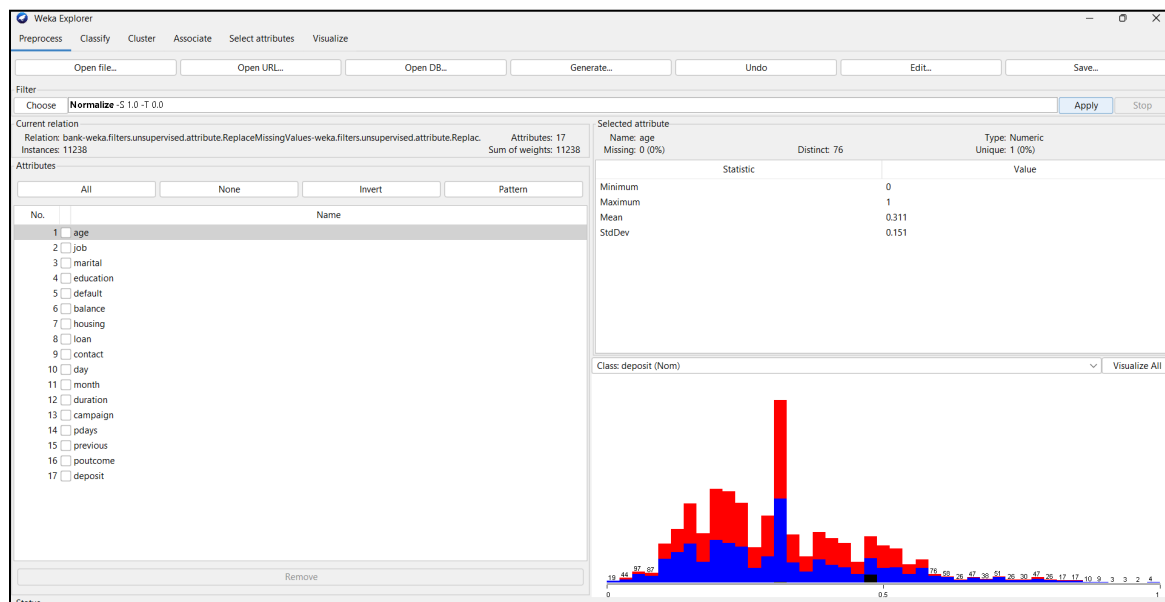
Pattern

No.	Name
1	☒ age
2	☐ job
3	☐ marital
4	☐ education
5	☐ default
6	☐ balance
7	☐ housing
8	☐ loan
9	☐ contact
10	☐ day
11	☐ month
12	☐ duration
13	☐ campaign
14	☐ pdays
15	☐ previous
16	☐ poutcome
17	☐ deposit

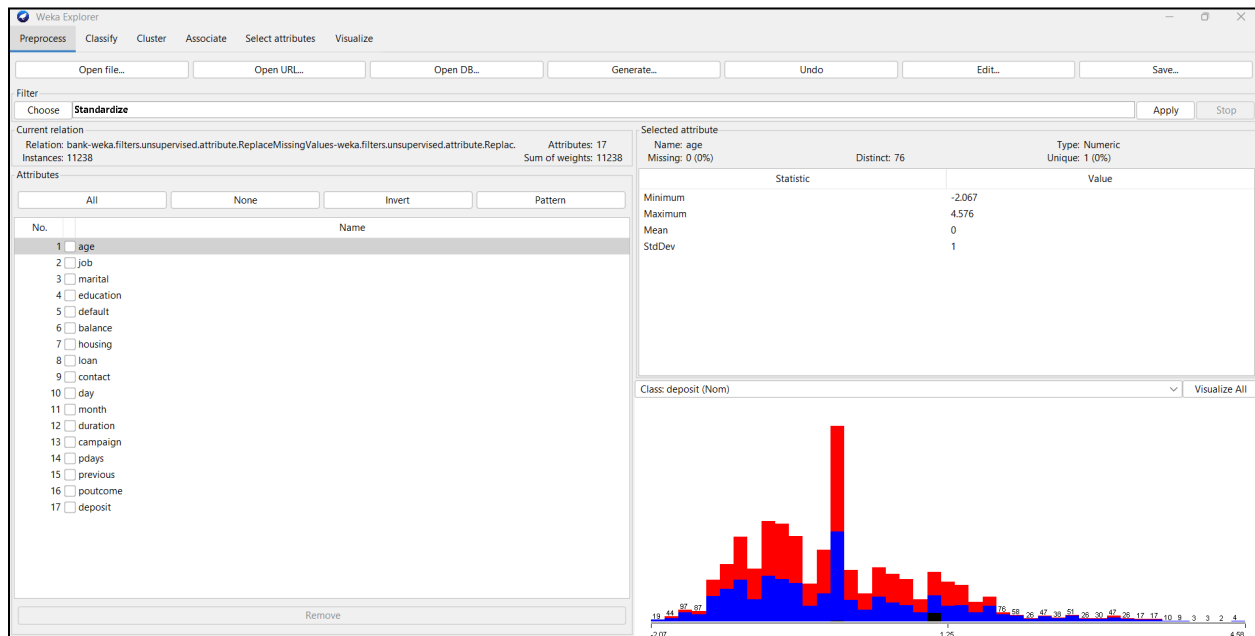
Remove

Normalization and Standardization

- Apply preprocessing filters:
 - **Normalize**: scales numeric values between 0 and 1.

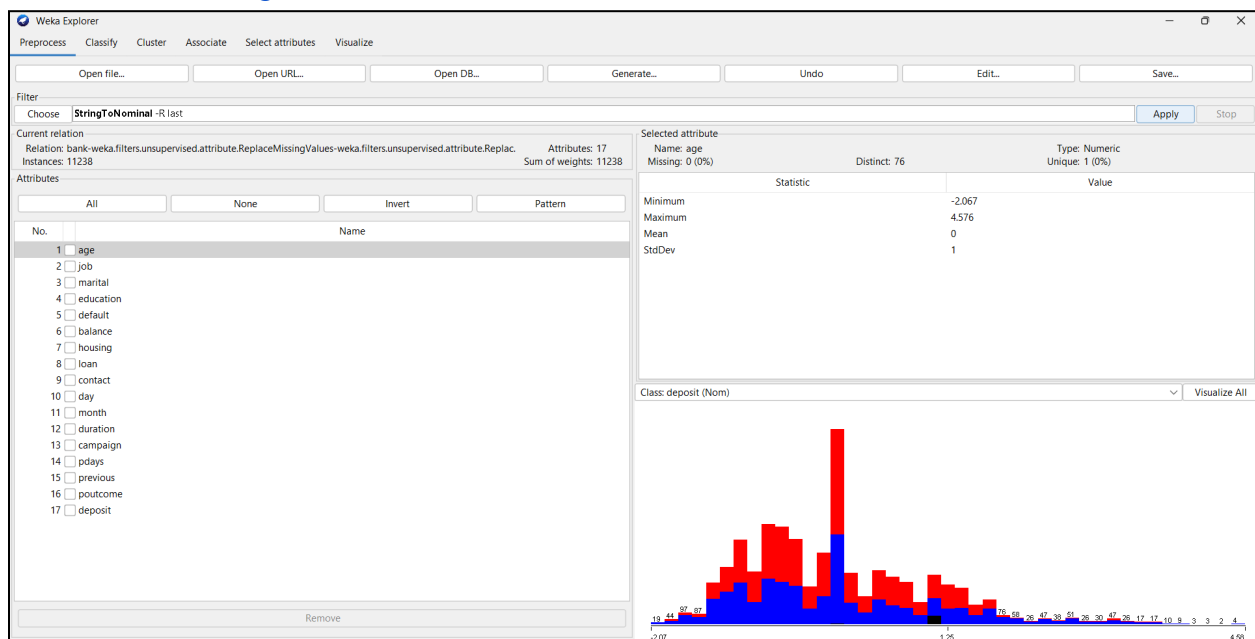


- **Standardize**: transforms data to zero mean and unit variance.



Converting Data Types

- Convert string attributes to nominal format using:
 - [StringToNominal](#).



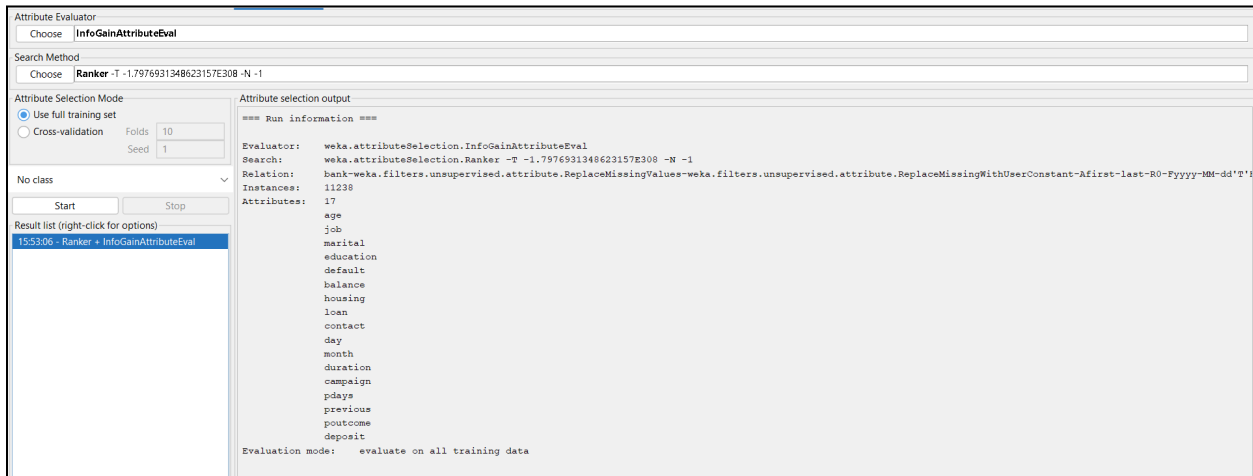
These transformations improve model performance and stability.

5. Feature Selection (Attribute Reduction)

Steps:

1. Go to the **Select attributes** tab.
2. Choose an attribute evaluator (for example, [InfoGainAttributeEval](#)).
3. Choose a search method (for example, [Ranker](#)).

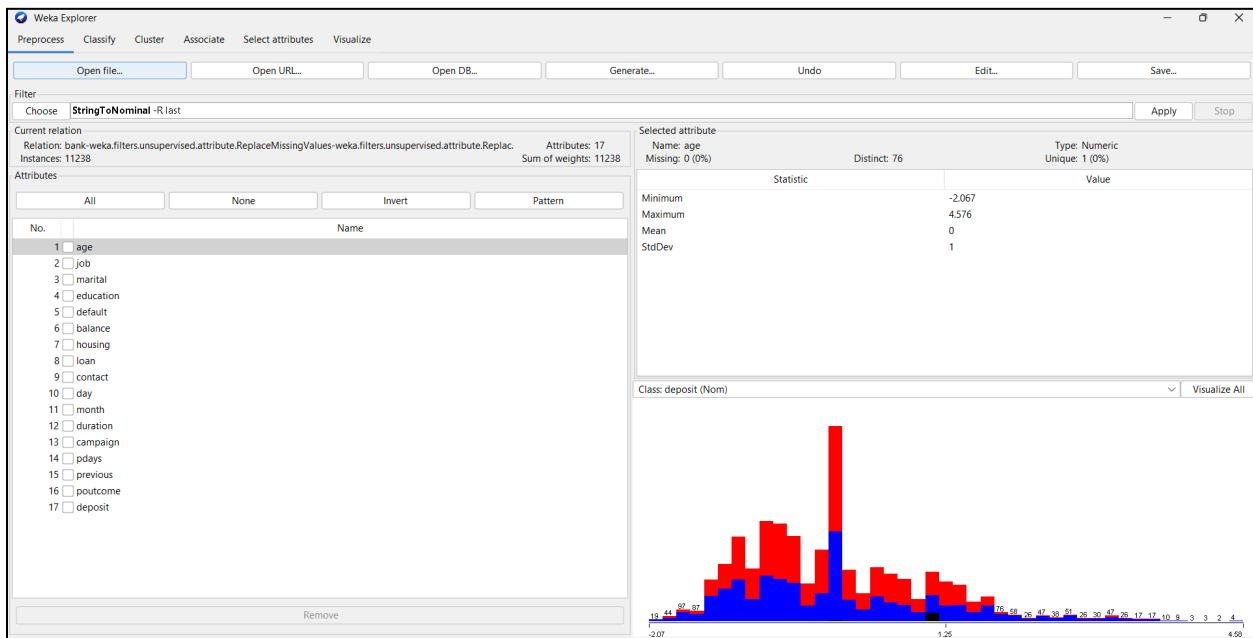
4. Click Start.



Outcome:

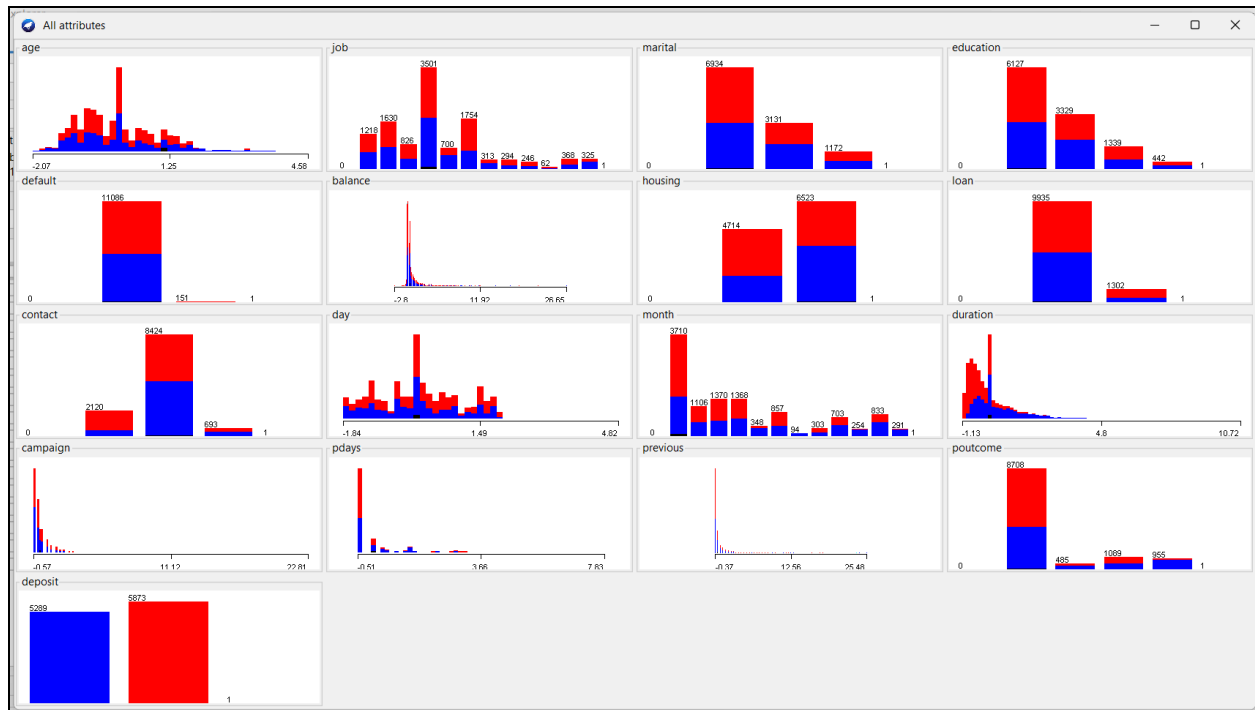
- Attributes are ranked based on their importance.
- Redundant and irrelevant attributes can be removed.

Feature selection reduces overfitting and improves learning efficiency.



6. Visual Data Inspection

- Click **Visualize All**.
- Scatter plots display relationships between attributes.
- Points are color-coded based on class labels.



Visual inspection helps detect outliers, clusters, and data patterns.

7. Final Preprocessed Dataset

After preprocessing:

- Missing values are handled.
- Attributes are cleaned and transformed.
- Data is normalized and optimized.

The dataset is now ready for classification, clustering, or regression tasks.

Exp 2: Implementation of Classification and Clustering Algorithms using WEKA

A) Classification using J48 Algorithm

1. Open the WEKA Tool.
2. Click on **Explorer**.
3. Go to the **Preprocess** tab.
4. Click **Open file** and load the dataset (.arff or .csv).
5. Select the **Classify** tab.
6. Click **Choose**.
7. Select:
 - trees
 - J48
8. Choose the test option:
 - Use training set / Percentage split / Cross validation.
9. Click **Start**.
10. Observe the output:

- Accuracy
- Confusion Matrix
- Correctly classified instances
- Error rate

Output:

Weka Explorer

Preprocess **Classify** Cluster Associate Select attributes Visualize

Classifier: Choose **BayesNet** -D -Q weka.classifiers.bayes.net.search.local.K2 -- -P 1 -S BAYES -E weka.classifiers.bayes.net.estimate.SimpleEstimator -- -A 0.5

Test options

☒ Use training set
☐ Supplied test set Set...
☐ Cross-validation Folds 10
☐ Percentage split % 66
 More options...

(Nom) Class

Start Stop

Result list (right-click for options)

11:44:15 - bayes.BayesNet

Classifier output

Time taken to build model: 0.01 seconds

=== Evaluation on training set ===

Time taken to test model on training data: 0 seconds

=== Summary ===

Correctly Classified Instances	217	75.8741 %
Incorrectly Classified Instances	69	24.1259 %
Kappa statistic	0.3958	
Mean absolute error	0.3018	
Root mean squared error	0.4284	
Relative absolute error	72.1442 %	
Root relative squared error	93.7317 %	
Total Number of Instances	286	

=== Detailed Accuracy By Class ===

	TP Rate	FP Rate	Precision	Recall	F-Measure	MCC	ROC Area	PRC Area	Class
Weighted Avg.	0.861	0.482	0.808	0.861	0.834	0.398	0.759	0.878	no-recurrence-events
	0.518	0.139	0.611	0.518	0.561	0.398	0.759	0.612	recurrence-events

=== Confusion Matrix ===

	a	b	<-- classified as
173 28	a = no-recurrence-events		
41 44	b = recurrence-events		

Status: OK Log x0

Weka Explorer

Preprocess **Classify** Cluster Associate Select attributes Visualize

Classifier: Choose **BayesNet** -D -Q weka.classifiers.bayes.net.search.local.K2 -- -P 1 -S BAYES -E weka.classifiers.bayes.net.estimate.SimpleEstimator -- -A 0.5

Test options

☐ Use training set
☐ Supplied test set Set...
☒ Cross-validation Folds 10
☐ Percentage split % 66
 More options...

(Nom) Class

Start Stop

Result list (right-click for options)

11:44:15 - bayes.BayesNet
 11:45:39 - bayes.BayesNet

Classifier output

LogScore ENTROPY: -2959.998063835574
 LogScore AIC: -3044.998063835574

Time taken to build model: 0 seconds

=== Stratified cross-validation ===

=== Summary ===

Correctly Classified Instances	206	72.028 %
Incorrectly Classified Instances	80	27.972 %
Kappa statistic	0.2919	
Mean absolute error	0.3297	
Root mean squared error	0.4566	
Relative absolute error	78.7898 %	
Root relative squared error	99.9047 %	
Total Number of Instances	286	

=== Detailed Accuracy By Class ===

	TP Rate	FP Rate	Precision	Recall	F-Measure	MCC	ROC Area	PRC Area	Class
Weighted Avg.	0.841	0.565	0.779	0.841	0.809	0.295	0.698	0.833	no-recurrence-events
	0.435	0.159	0.536	0.435	0.481	0.295	0.698	0.510	recurrence-events

=== Confusion Matrix ===

	a	b	<-- classified as
169 32	a = no-recurrence-events		
48 37	b = recurrence-events		

Status: OK Log x0

Weka Explorer

Preprocess **Classify** Cluster Associate Select attributes Visualize

Classifier
Choose **BayesNet** -D -Q weka.classifiers.bayes.net.search.local.K2 --P 1 -S BAYES-E weka.classifiers.bayes.net.estimate.SimpleEstimator --A 0.5

Test options
☐ Use training set
☐ Supplied test set Set...
☐ Cross-validation Folds 10
☒ Percentage split % 66
More options...

(Nom) Class
Start Stop

Result list (right-click for options)
11:44:15 - bayes.BayesNet
11:45:39 - bayes.BayesNet
11:46:10 - bayes.BayesNet

Status
OK

Classifier output

Time taken to build model: 0.01 seconds

=== Evaluation on test split ===

Time taken to test model on test split: 0 seconds

=== Summary ===

Correctly Classified Instances	68	70.1031 %
Incorrectly Classified Instances	29	29.8969 %
Kappa statistic	0.3087	
Mean absolute error	0.3474	
Root mean squared error	0.4902	
Relative absolute error	81.0012 %	
Root relative squared error	102.5618 %	
Total Number of Instances	97	

=== Detailed Accuracy By Class ===

	TP Rate	FP Rate	Precision	Recall	F-Measure	MCC	ROC Area	PRC Area	Class
	0.813	0.515	0.754	0.813	0.782	0.311	0.659	0.778	no-recurrence-events
	0.485	0.188	0.571	0.485	0.525	0.311	0.659	0.543	recurrence-events
Weighted Avg.	0.701	0.404	0.692	0.701	0.694	0.311	0.659	0.698	

=== Confusion Matrix ===

a b <-- classified as

52 12 | a = no-recurrence-events

17 16 | b = recurrence-events

Log x0

Weka Explorer

Preprocess **Classify** Cluster Associate Select attributes Visualize

Classifier
Choose **NaiveBayes**

Test options
☒ Use training set
☐ Supplied test set Set...
☐ Cross-validation Folds 10
☐ Percentage split % 66
More options...

(Nom) Class
Start Stop

Result list (right-click for options)
11:44:15 - bayes.BayesNet
11:45:39 - bayes.BayesNet
11:46:10 - bayes.BayesNet
11:53:21 - bayes.NaiveBayes

Status
OK

Classifier output

Time taken to build model: 0.02 seconds

=== Evaluation on training set ===

Time taken to test model on training data: 0 seconds

=== Summary ===

Correctly Classified Instances	215	75.1748 %
Incorrectly Classified Instances	71	24.8252 %
Kappa statistic	0.3693	
Mean absolute error	0.3012	
Root mean squared error	0.4278	
Relative absolute error	72.0082 %	
Root relative squared error	93.6095 %	
Total Number of Instances	286	

=== Detailed Accuracy By Class ===

	TP Rate	FP Rate	Precision	Recall	F-Measure	MCC	ROC Area	PRC Area	Class
	0.866	0.518	0.798	0.866	0.831	0.374	0.760	0.879	no-recurrence-events
	0.482	0.134	0.603	0.482	0.536	0.374	0.760	0.610	recurrence-events
Weighted Avg.	0.752	0.404	0.740	0.752	0.743	0.374	0.760	0.799	

=== Confusion Matrix ===

a b <-- classified as

174 27 | a = no-recurrence-events

44 41 | b = recurrence-events

Log x0

Weka Explorer

Preprocess

Classify

Cluster

Associate

Select attributes

Visualize

Classifier

Choose NaiveBayesMultinomialText -P 0 -M 3.0 -norm 1.0 -lnorm 2.0 -stopwords-handler weka.core.stopwords.Null -tokenizer "weka.core.tokenizers.WordTokenizer -delimiters '\n\r\n\t,;\|\\|'0?" -stemmer weka.core.stemmers.NullStemmer

Test options

☒ Use training set

Set...

☐ Supplied test set

Set...

☐ Cross-validation

Folds 10

☐ Percentage split

% 66

More options...

(Nom) Class

Start

Stop

Result list (right-click for options)

11:44:15 - bayes.BayesNet

11:45:39 - bayes.BayesNet

11:46:10 - bayes.BayesNet

11:53:21 - bayes.NaiveBayes

11:54:00 - bayes.NaiveBayes

11:54:23 - bayes.NaiveBayes

11:54:49 - bayes.NaiveBayesMultinomialText

Classifier output

Time taken to build model: 0 seconds

=== Evaluation on training set ===

Time taken to test model on training data: 0 seconds

=== Summary ===

Correctly Classified Instances	201	70.2797 %
Incorrectly Classified Instances	85	29.7203 %
Kappa statistic	0	
Mean absolute error	0.4183	
Root mean squared error	0.457	
Relative absolute error	100 %	
Root relative squared error	100 %	
Total Number of Instances	286	

=== Detailed Accuracy By Class ===

	TP Rate	FP Rate	Precision	Recall	F-Measure	MCC	ROC Area	PRC Area	Class
	1.000	1.000	0.703	1.000	0.825	?	0.500	0.703	no-recurrence-events
	0.000	0.000	?	0.000	?	?	0.500	0.297	recurrence-events
Weighted Avg.	0.703	0.703	?	0.703	?	?	0.500	0.582	

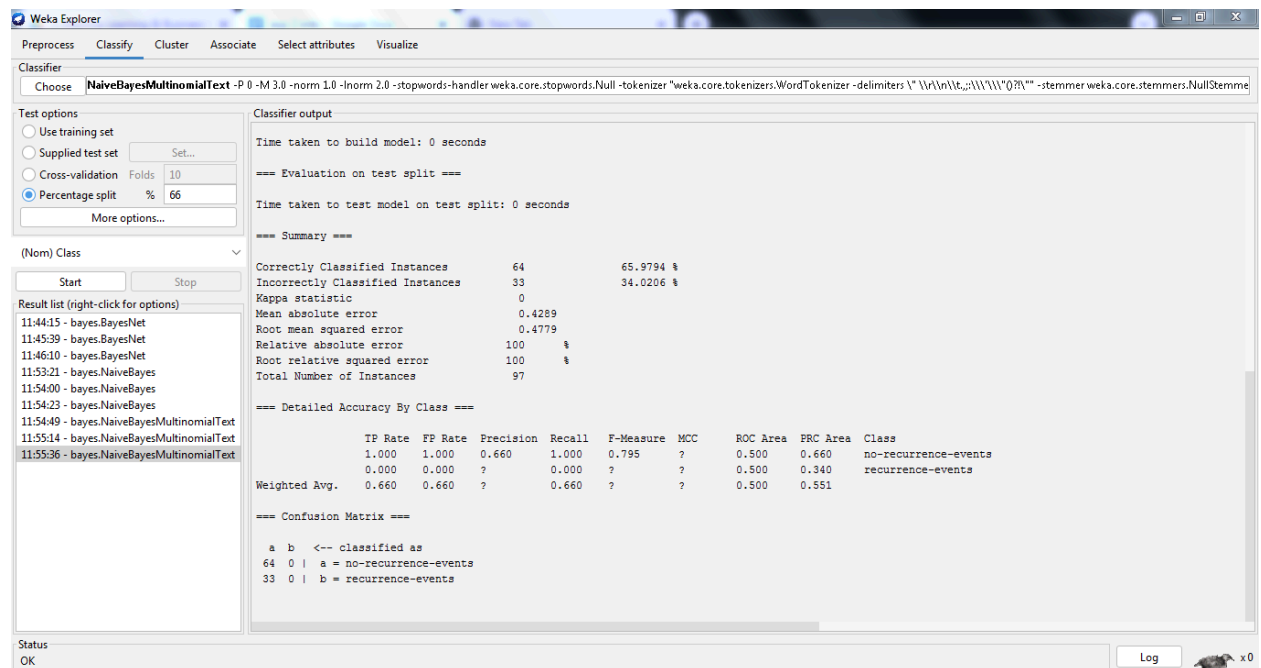
=== Confusion Matrix ===

a	b	<-- classified as	
201	0	a = no-recurrence-events	
85	0	b = recurrence-events	

Status

OK

Log



4. Select:

- bayes
- NaiveBayes

5. Select the testing option.

6. Click **Start**.

Output:

Weka Explorer

Preprocess **Classify** Cluster Associate Select attributes Visualize

Classifier: Choose **NaiveBayesUpdateable**

Test options:

- ☒ Use training set
- ☐ Supplied test set (Set...)
- ☐ Cross-validation (Folds: 10)
- ☐ Percentage split (%: 66)
- More options...

(Nom) Class: Start Stop

Result list (right-click for options):

- 11:44:15 - bayes.BayesNet
- 11:45:39 - bayes.BayesNet
- 11:46:10 - bayes.BayesNet
- 11:53:21 - bayes.NaiveBayes
- 11:54:00 - bayes.NaiveBayes
- 11:54:23 - bayes.NaiveBayes
- 11:54:49 - bayes.NaiveBayesMultinomialText
- 11:55:14 - bayes.NaiveBayesMultinomialText
- 11:55:36 - bayes.NaiveBayesMultinomialText
- 11:56:34 - bayes.NaiveBayesUpdateable

Classifier output:

Time taken to build model: 0 seconds

=== Evaluation on training set ===

Time taken to test model on training data: 0 seconds

=== Summary ===

Correctly Classified Instances	215	75.1748 %
Incorrectly Classified Instances	71	24.8252 %
Kappa statistic	0.3693	
Mean absolute error	0.3012	
Root mean squared error	0.4278	
Relative absolute error	72.0082 %	
Root relative squared error	93.6095 %	
Total Number of Instances	286	

=== Detailed Accuracy By Class ===

	TP Rate	FP Rate	Precision	Recall	F-Measure	MCC	ROC Area	PRC Area	Class
Weighted Avg.	0.866	0.518	0.798	0.866	0.831	0.374	0.760	0.879	no-recurrence-events
	0.482	0.134	0.603	0.482	0.536	0.374	0.760	0.610	recurrence-events

=== Confusion Matrix ===

```

a  b  <-- classified as
174 27 | a = no-recurrence-events
 44 41 | b = recurrence-events

```

Status: OK

Weka Explorer

Preprocess **Classify** Cluster Associate Select attributes Visualize

Classifier: Choose **NaiveBayesUpdateable**

Test options:

- ☐ Use training set
- ☐ Supplied test set (Set...)
- ☒ Cross-validation (Folds: 10)
- ☐ Percentage split (%: 66)
- More options...

(Nom) Class: Start Stop

Result list (right-click for options):

- 11:44:15 - bayes.BayesNet
- 11:45:39 - bayes.BayesNet
- 11:46:10 - bayes.BayesNet
- 11:53:21 - bayes.NaiveBayes
- 11:54:00 - bayes.NaiveBayes
- 11:54:23 - bayes.NaiveBayes
- 11:54:49 - bayes.NaiveBayesMultinomialText
- 11:55:14 - bayes.NaiveBayesMultinomialText
- 11:55:36 - bayes.NaiveBayesMultinomialText
- 11:56:34 - bayes.NaiveBayesUpdateable

Classifier output:

[total] 209.0 87.0

Time taken to build model: 0.01 seconds

=== Stratified cross-validation ===

=== Summary ===

Correctly Classified Instances	205	71.6783 %
Incorrectly Classified Instances	81	28.3217 %
Kappa statistic	0.2857	
Mean absolute error	0.3272	
Root mean squared error	0.4534	
Relative absolute error	78.2086 %	
Root relative squared error	99.1872 %	
Total Number of Instances	286	

=== Detailed Accuracy By Class ===

	TP Rate	FP Rate	Precision	Recall	F-Measure	MCC	ROC Area	PRC Area	Class
Weighted Avg.	0.836	0.565	0.778	0.836	0.806	0.288	0.701	0.837	no-recurrence-events
	0.435	0.164	0.529	0.435	0.477	0.288	0.701	0.514	recurrence-events

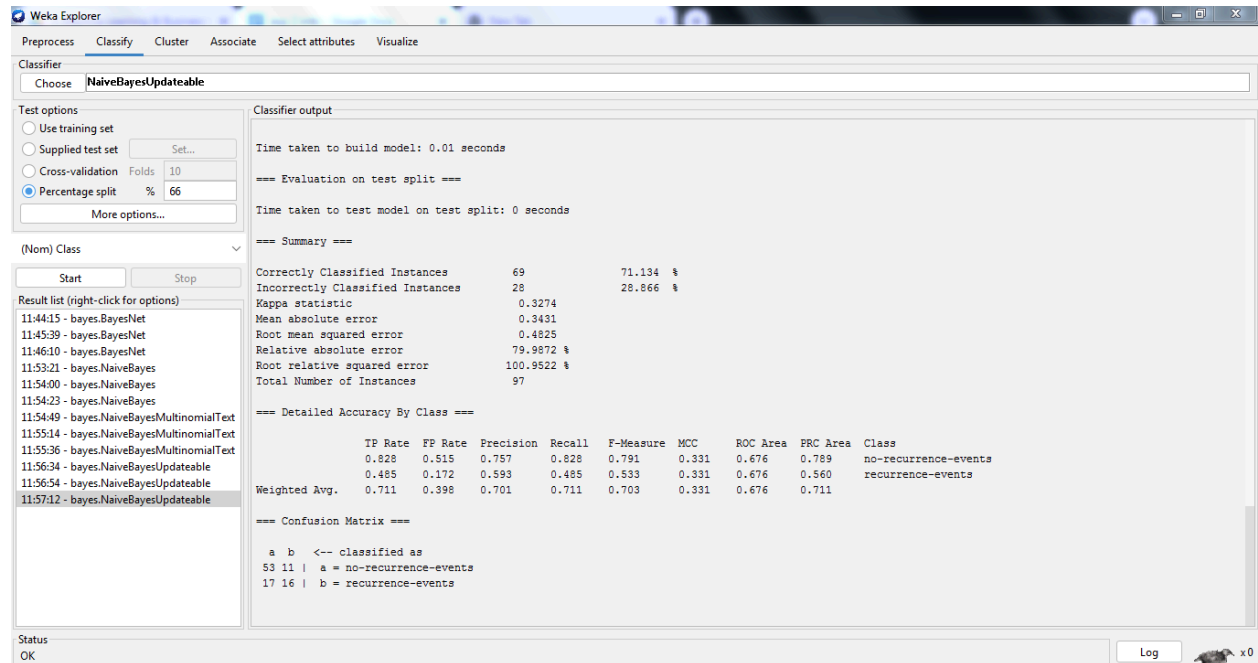
=== Confusion Matrix ===

```

a  b  <-- classified as
168 33 | a = no-recurrence-events
 48 37 | b = recurrence-events

```

Status: OK



C) Clustering using SimpleKMeans

1. Open WEKA Explorer.
2. Load the dataset.
3. Go to the **Cluster** tab.
4. Click **Choose**.
5. Select:
 - o SimpleKMeans
6. Set number of clusters (K value).
7. Click **Start**.

Output:

Weka Explorer

Preprocess Classify **Cluster** Associate Select attributes Visualize

Clusterer
Choose **Canopy -N -1 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t2 -1.0 -t1 -1.25 -S1**

Cluster mode
☒ Use training set
☐ Supplied test set Set...
☐ Percentage split % 66
☐ Classes to clusters evaluation
 (Nom) Class ▾
☒ Store clusters for visualization

Ignore attributes
Start Stop

Result list (right-click for options)
 11:58:45 - Canopy

Clusterer output

17	10	(3%)
18	5	(2%)
19	6	(2%)
20	7	(2%)
21	6	(2%)
22	4	(1%)
23	2	(1%)
24	4	(1%)
25	4	(1%)
26	9	(3%)
27	4	(1%)
28	5	(2%)
29	2	(1%)
30	4	(1%)
31	4	(1%)
32	3	(1%)
33	4	(1%)
34	2	(1%)
35	2	(1%)
36	10	(3%)
37	3	(1%)
38	5	(2%)
39	2	(1%)
40	4	(1%)
41	5	(2%)
42	2	(1%)
43	1	(0%)
44	2	(1%)
45	6	(2%)
46	6	(2%)

Status
OK

Log x0

Weka Explorer

Preprocess Classify **Cluster** Associate Select attributes Visualize

Clusterer
Choose **Canopy -N -1 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t2 -1.0 -t1 -1.25 -S1**

Cluster mode
☐ Use training set
☐ Supplied test set Set...
☒ Percentage split % 66
☐ Classes to clusters evaluation
 (Nom) Class ▾
☒ Store clusters for visualization

Ignore attributes
Start Stop

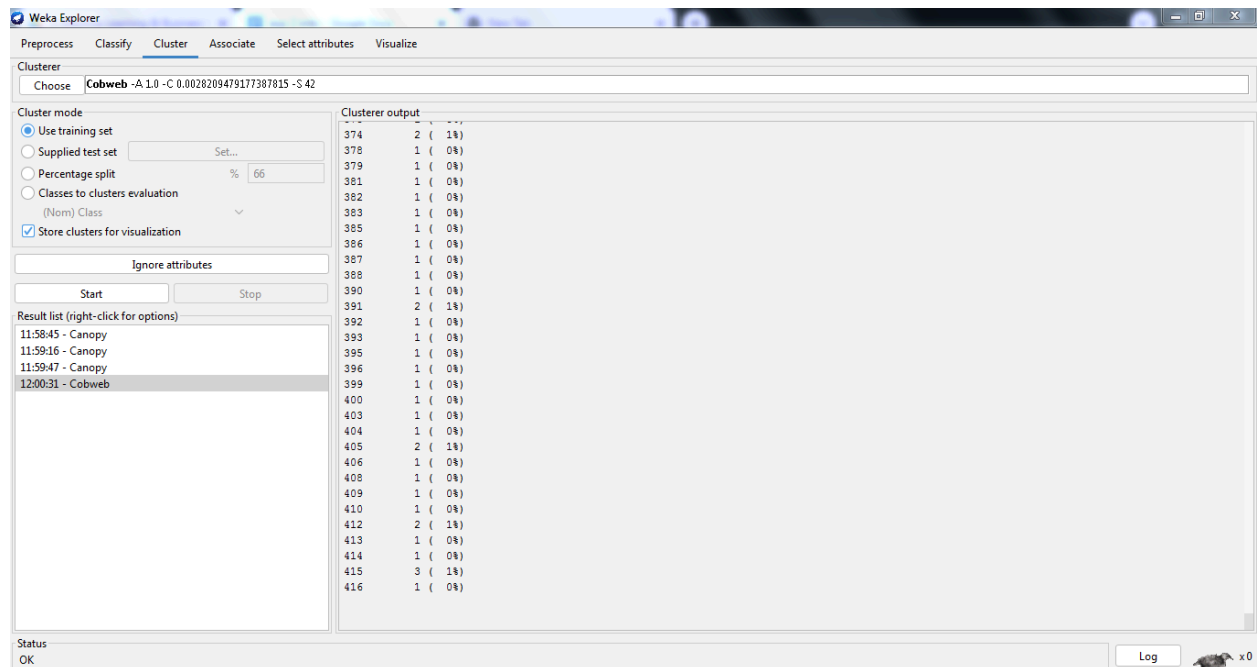
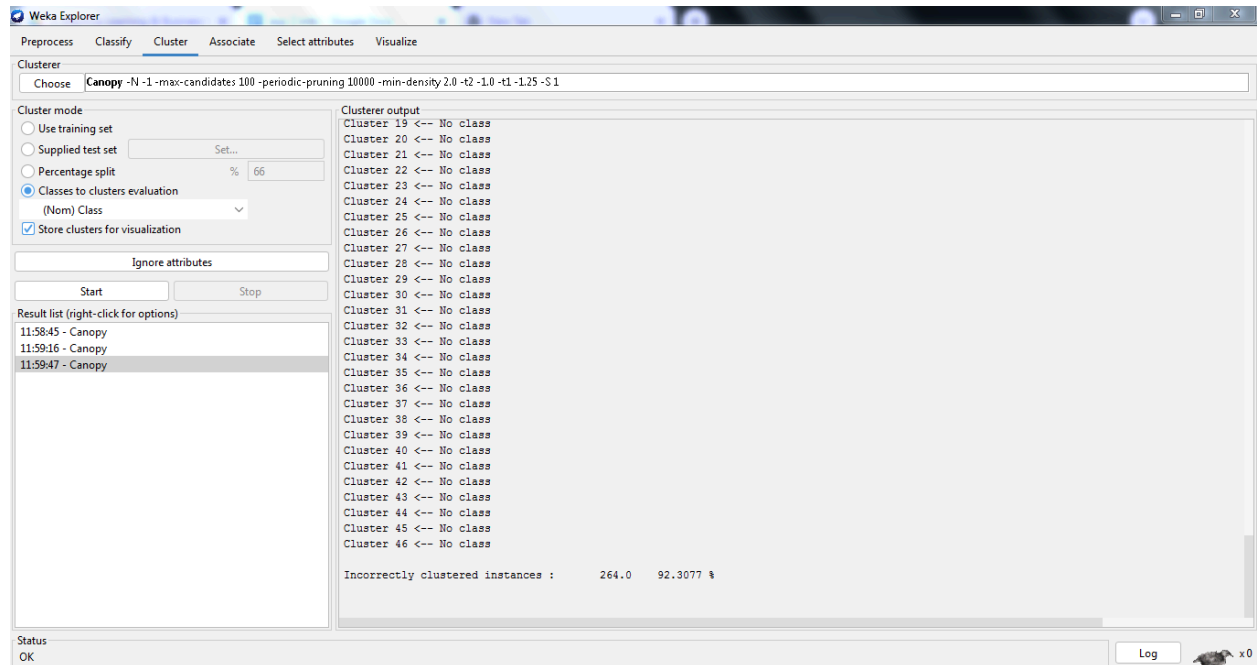
Result list (right-click for options)
 11:58:45 - Canopy
 11:59:16 - Canopy

Clusterer output

5	4	(4%)
6	6	(6%)
7	4	(4%)
8	4	(4%)
9	3	(3%)
10	4	(4%)
11	3	(3%)
12	4	(4%)
13	1	(1%)
14	5	(5%)
15	3	(3%)
16	1	(1%)
17	2	(2%)
18	1	(1%)
20	3	(3%)
21	4	(4%)
22	2	(2%)
23	1	(1%)
24	2	(2%)
25	3	(3%)
26	3	(3%)
28	1	(1%)
29	1	(1%)
30	1	(1%)
32	1	(1%)
33	2	(2%)
34	1	(1%)
35	1	(1%)
36	1	(1%)
40	1	(1%)

Status
OK

Log x0



Weka Explorer

Preprocess Classify Cluster Associate Select attributes Visualize

Clusterer

Choose Cobweb -A 1.0 -C 0.0028209479177387815 -S 42

Cluster mode

☐ Use training set

☐ Supplied test set Set...

☒ Percentage split % 66

☐ Classes to clusters evaluation (Nom) Class

☒ Store clusters for visualization

Ignore attributes

Start Stop

Result list (right-click for options)

11:58:45 - Canopy

11:59:16 - Canopy

11:59:47 - Canopy

12:00:31 - Cobweb

Cluster output

374 2 (1%)

378 1 (0%)

379 1 (0%)

381 1 (0%)

382 1 (0%)

383 1 (0%)

385 1 (0%)

386 1 (0%)

387 1 (0%)

388 1 (0%)

390 1 (0%)

391 2 (1%)

392 1 (0%)

393 1 (0%)

395 1 (0%)

396 1 (0%)

399 1 (0%)

400 1 (0%)

403 1 (0%)

404 1 (0%)

405 2 (1%)

406 1 (0%)

408 1 (0%)

409 1 (0%)

410 1 (0%)

412 2 (1%)

413 1 (0%)

414 1 (0%)

415 3 (1%)

416 1 (0%)

Status OK

Log x0

Weka Explorer

Preprocess Classify Cluster Associate Select attributes Visualize

Clusterer

Choose Cobweb -A 1.0 -C 0.0028209479177387815 -S 42

Cluster mode

☐ Use training set

☐ Supplied test set Set...

☐ Percentage split % 66

☒ Classes to clusters evaluation (Nom) Class

☒ Store clusters for visualization

Ignore attributes

Start Stop

Result list (right-click for options)

11:58:45 - Canopy

11:59:16 - Canopy

11:59:47 - Canopy

12:00:31 - Cobweb

12:01:29 - Cobweb

Cluster output

Cluster 362 <-- No class

Cluster 364 <-- No class

Cluster 365 <-- No class

Cluster 367 <-- recurrence-events

Cluster 368 <-- No class

Cluster 370 <-- No class

Cluster 371 <-- No class

Cluster 372 <-- No class

Cluster 373 <-- No class

Cluster 376 <-- No class

Cluster 377 <-- No class

Cluster 379 <-- No class

Cluster 380 <-- No class

Cluster 383 <-- No class

Cluster 384 <-- No class

Cluster 385 <-- No class

Cluster 387 <-- No class

Cluster 388 <-- No class

Cluster 389 <-- No class

Cluster 390 <-- No class

Cluster 393 <-- No class

Cluster 394 <-- No class

Cluster 395 <-- No class

Cluster 398 <-- No class

Cluster 401 <-- No class

Cluster 402 <-- No class

Cluster 403 <-- No class

Cluster 404 <-- No class

Incorrectly clustered instances : 279.0 97.5524 %

Status OK

Log x0

Weka Explorer

Preprocess Classify **Cluster** Associate Select attributes Visualize

Clusterer
Choose **EM** -I 100 -N -1 -X 10 -max -1 -ll-cv 1.0E-6 -ll-iter 1.0E-6 -M 1.0E-6 -K 10 -num-slots 1 -S 100

Cluster mode
☒ Use training set
☐ Supplied test set Set...
☐ Percentage split % 66
☐ Classes to clusters evaluation (Nom) Class
☒ Store clusters for visualization

Ignore attributes
Start Stop

Result list (right-click for options)
 11:58:45 - Canopy
 11:59:16 - Canopy
 11:59:47 - Canopy
 12:00:31 - Cobweb
 12:01:29 - Cobweb
 12:02:39 - Cobweb
 12:03:19 - **EM**

Clusterer output

[total]	104.9785	117.3751	69.6463
breast-quad			
left_up	42.386	34.7548	22.8593
left_low	39.6159	45.9394	28.4448
right_up	10.4952	13.9286	11.5761
right_low	4.7102	15.7882	6.5016
central	10.7714	9.9641	3.2646
[total]	107.9785	120.3751	72.6463
irradiat			
yes	14.4847	18.5795	37.9357
no	90.4938	98.7956	31.7106
[total]	104.9785	117.3751	69.6463
Class			
no-recurrence-events	86.87	89.3144	27.8156
recurrence-events	18.1085	28.0608	41.8307
[total]	104.9785	117.3751	69.6463

Time taken to build model (full training data) : 0.75 seconds

=== Model and evaluation on training set ===

Clustered Instances

0	104	(36%)
1	117	(41%)
2	65	(23%)

Log likelihood: -9.36546

Status OK Log x0

Weka Explorer

Preprocess Classify **Cluster** Associate Select attributes Visualize

Clusterer
Choose **EM** -I 100 -N -1 -X 10 -max -1 -ll-cv 1.0E-6 -ll-iter 1.0E-6 -M 1.0E-6 -K 10 -num-slots 1 -S 100

Cluster mode
☐ Use training set
☐ Supplied test set Set...
☒ Percentage split % 66
☐ Classes to clusters evaluation (Nom) Class
☒ Store clusters for visualization

Ignore attributes
Start Stop

Result list (right-click for options)
 11:58:45 - Canopy
 11:59:16 - Canopy
 11:59:47 - Canopy
 12:00:31 - Cobweb
 12:01:29 - Cobweb
 12:02:39 - Cobweb
 12:03:19 - **EM**
 12:03:38 - **EM**

Clusterer output

left	37.4677	22.3001	40.2322
right	29.1667	26.8581	37.9752
[total]	66.6344	49.1582	78.2074
breast-quad			
left_up	26.8404	19.4599	15.6997
left_low	23.65	15.5672	35.7828
right_up	7.0388	8.0152	9.9461
right_low	3.2639	5.7932	10.9429
central	8.8413	3.3226	8.836
[total]	69.6344	52.1582	81.2074
irradiat			
yes	7.141	29.2188	13.6402
no	59.4934	19.9395	64.5672
[total]	66.6344	49.1582	78.2074
Class			
no-recurrence-events	58.2202	20.5879	60.1918
recurrence-events	8.4141	28.5703	18.0156
[total]	66.6344	49.1582	78.2074

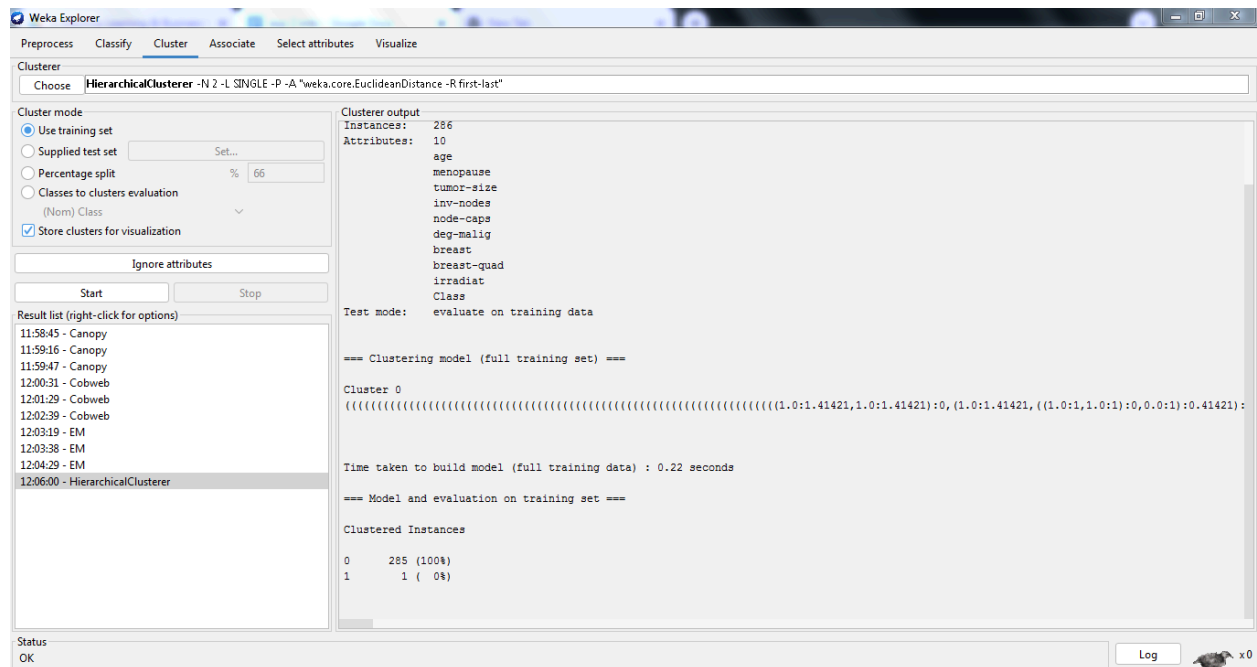
Time taken to build model (percentage split) : 0.31 seconds

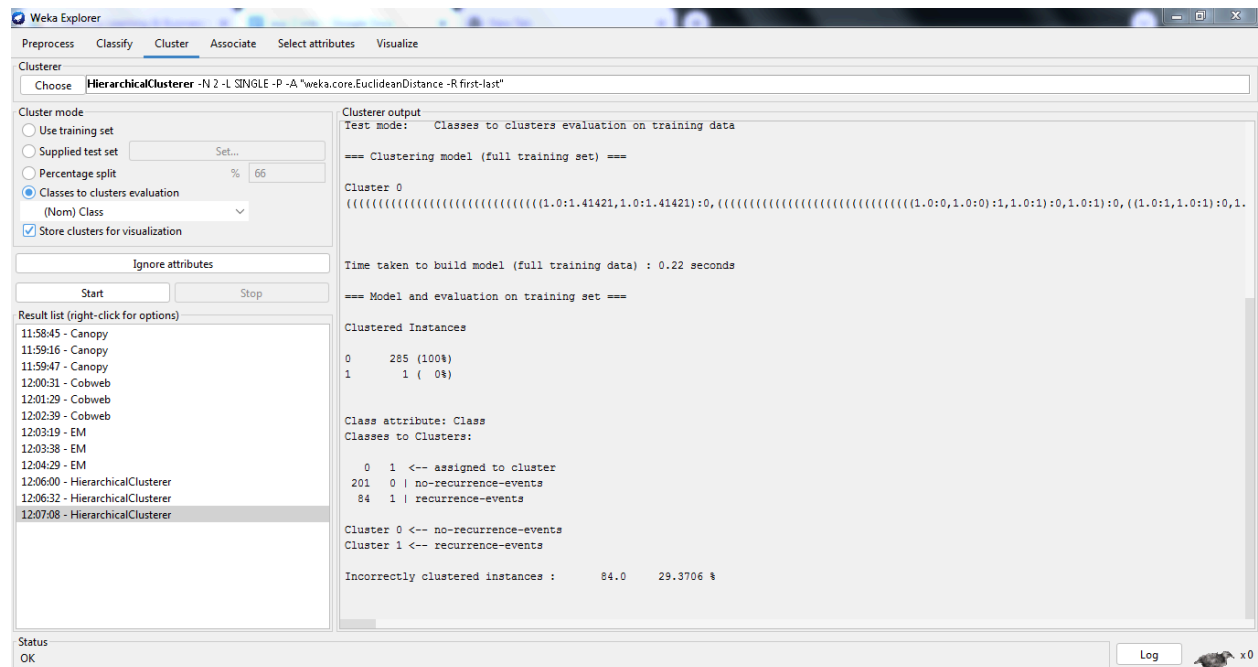
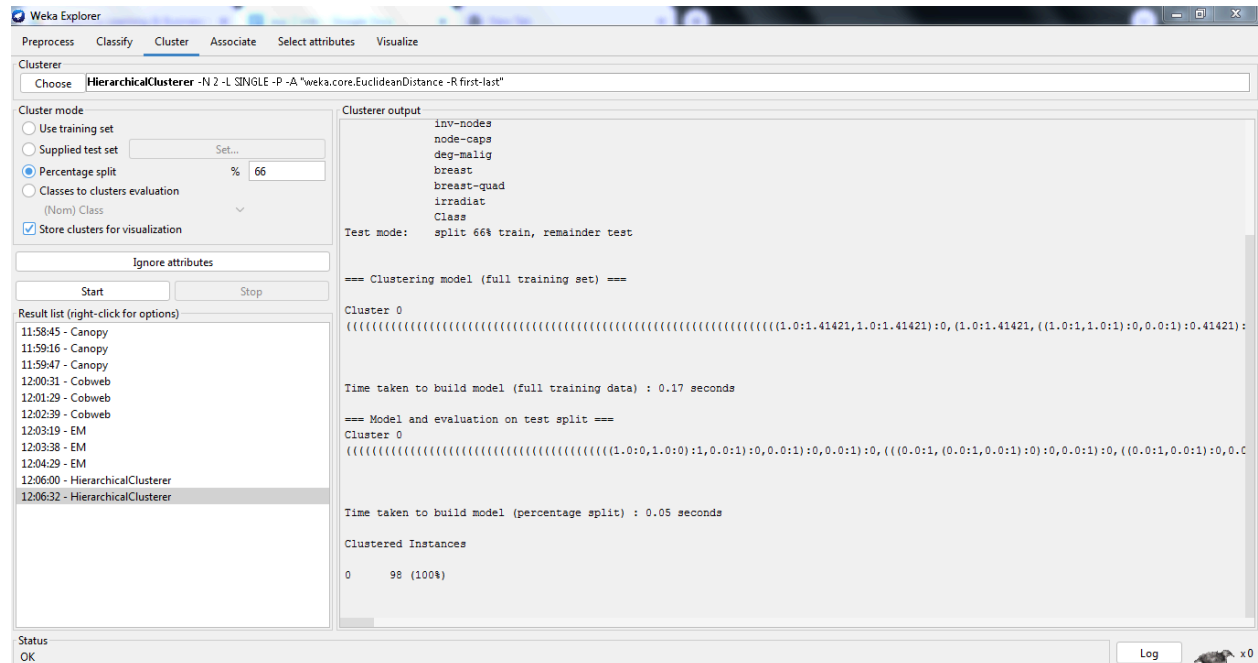
Clustered Instances

0	35	(36%)
1	26	(27%)
2	37	(38%)

Log likelihood: -9.77657

Status OK Log x0





Exp 3: Implementation of Decision Tree Algorithm using Python.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
```

```

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = {
    "Company": [
        "Google", "Amazon", "TCS", "Infosys", "Microsoft",
        "Google", "Amazon", "TCS", "Microsoft", "Infosys"
    ],
    "Degree": [
        "BTech", "MTech", "BSc", "BTech", "MTech",
        "PhD", "BTech", "MTech", "BTech", "MSc"
    ],
    "Role": [
        "Data Scientist", "Software Engineer", "Developer", "Analyst", "ML Engineer",
        "Research Scientist", "SDE", "Developer", "Architect", "Analyst"
    ],
    "Salary_Above_30LPA": [1, 1, 0, 0, 1, 1, 1, 0, 1, 0]
}

df = pd.DataFrame(data)
X = df[["Company", "Degree", "Role"]]
y = df["Salary_Above_30LPA"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

categorical_features = ["Company", "Degree", "Role"]
preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ]
)

model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", DecisionTreeClassifier(criterion="entropy", random_state=42))
])

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

new_candidate = pd.DataFrame({
    "Company": ["Google"],
    "Degree": ["MTech"],
    "Role": ["ML Engineer"]
})

prediction = model.predict(new_candidate)
print("\nSalary Above 30 LPA (1=Yes, 0=No):", prediction[0])

```

Exp 4 : Implementation of Naive Bayes and Support Vector Machine (SVM)

Algorithms using Python

Naive Bayes:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report
# 1. Load CSV file (use your correct path)
data = pd.read_csv("Naive_based_Classification_problem_csv.csv")
# 2. Separate features and target
X = data.iloc[:, :-1] # features
y = data.iloc[:, -1]  # class label
# 3. Convert categorical features to numeric using One-Hot Encoding
X = pd.get_dummies(X)
# 4. Ensure all feature values are numeric
X = X.apply(pd.to_numeric)
# 5. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# 6. Create and train Naive Bayes model
nb = GaussianNB()
nb.fit(X_train, y_train)
# 7. Predict
y_pred = nb.predict(X_test)
# 8. Evaluate
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

SVM:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.preprocessing import StandardScaler
# Load dataset
data = pd.read_csv(r'Downloads/Naive_based_Classification_problem_csv.csv')
# Features and target
X = data.iloc[:, :-1]
y = data.iloc[:, -1]
# One-hot encoding
X = pd.get_dummies(X)
# Convert to numeric
X = X.apply(pd.to_numeric)
```

```

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# SVM model
svm = SVC(kernel='linear')
# Train
svm.fit(X_train, y_train)
# Predict
y_pred = svm.predict(X_test)
# Evaluate
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n",
      classification_report(y_test, y_pred, zero_division=0))

```

Exp 5: Implement Linear Regression/Logistic Regression algorithms using Python on real-world dataset

Linear Regression:

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
# Load dataset
data = fetch_california_housing()
X = data.data[:, 0].reshape(-1, 1)
y = data.target
# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2
)
# Train model
model = LinearRegression()
model.fit(X_train, y_train)
# Predictions
y_pred = model.predict(X_test)
# Evaluate

```

```

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
# Plot regression line
plt.scatter(X_test, y_test, label="Actual Data")
plt.plot(X_test, y_pred, label="Regression Line")
plt.title("Linear Regression: Actual vs Predicted")
plt.xlabel("Feature")
plt.ylabel("House Price")
plt.legend()
plt.show()

```

Logistic Regression:

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load dataset (use only 2 features for plotting)
data = load_breast_cancer()
X = data.data[:, :2]
y = data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Train model
model = LogisticRegression(max_iter=5000)
model.fit(X_train, y_train)

# Plot decision boundary
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.01),
                     np.arange(y_min, y_max, 0.01))

Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

# Make predictions on the test set for evaluation
y_pred = model.predict(X_test)

# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
plt.contourf(xx, yy, Z, alpha=0.3)
plt.title("Logistic Regression Decision Boundary")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()

```


Exp 6: Implementation of K-Means Clustering Algorithm using Python.

```
from sklearn.cluster import KMeans
import numpy as np
import matplotlib.pyplot as plt
X = np.array([
    [1,2], [1.5,1.8], [5,8],
    [8,8], [1,0.6], [9,11],
    [8,2], [10,2], [9,3]
])
kmeans = KMeans(n_clusters=3)
kmeans.fit(X)
centroids = kmeans.cluster_centers_
labels = kmeans.labels_
plt.scatter(X[:,0], X[:,1], c=labels, cmap='rainbow')
plt.scatter(centroids[:,0], centroids[:,1], s=200, c='black', marker='X')
plt.show()
```

Exp 7: Implementation of Association Rule Mining (Apriori Algorithm) using WEKA

Step 1: Create Dataset File

1. Open Notepad.
2. Type the dataset:

```
@relation market

@attribute bread {yes,no}
@attribute butter {yes,no}
@attribute jam {yes,no}
@attribute milk {yes,no}
@attribute juice {yes,no}
@attribute curd {yes,no}

@data
yes,yes,yes,yes,no,no
yes,yes,no,yes,no,no
yes,no,no,no,yes,yes
yes,no,no,yes,yes,no
no,yes,no,yes,yes,no
```

3. Save the file as:
market.arff

Step 2: Open WEKA

1. Open the **WEKA Tool**.
2. Click **Explorer**.

Step 3: Load Dataset

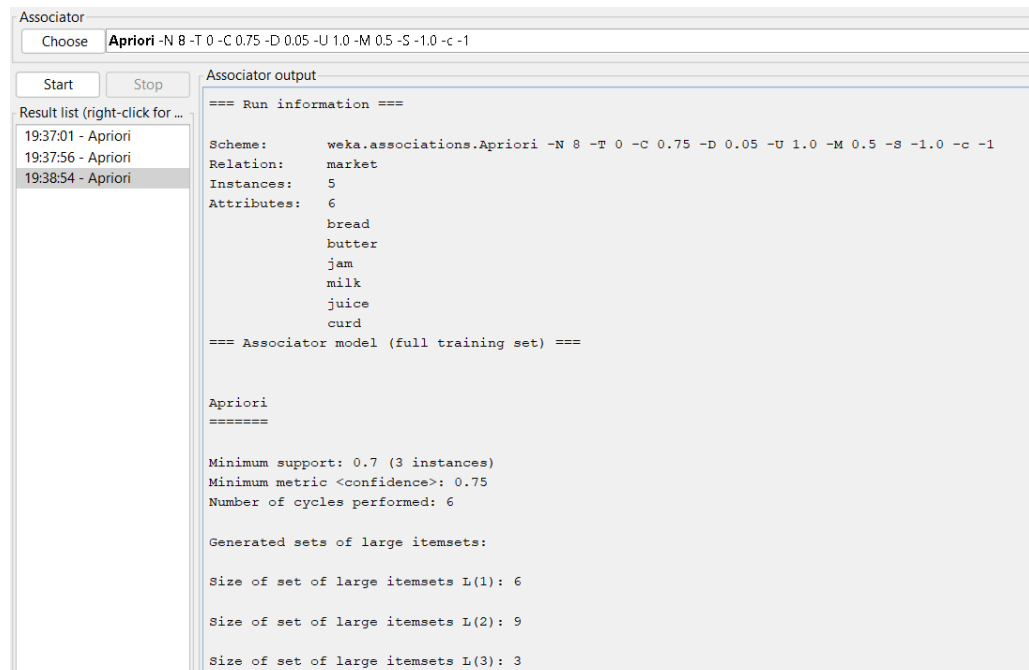
1. Go to the **Preprocess** tab.
2. Click **Open File**.
3. Select:

market.arff

Step 4: Select Apriori Algorithm

1. Go to the **Associate** tab.
2. Click **Choose**.
3. Select:

Associations → Apriori



Step 5: Run the Program

1. Set support and confidence values if needed.
2. Click **Start**.

Step 6: Observe Output

- Frequent itemsets generated
- Association rules displayed
- Support, confidence, and lift values shown

Associator

Choose **Apriori** -N 8 -T 0 -C 0.75 -D 0.05 -U 1.0 -M 0.5 -S -1.0 -c -1

Start Stop

Result list (right-click for ...)

- 19:37:01 - Apriori
- 19:37:56 - Apriori
- 19:38:54 - Apriori

Associator output

Instances: 5

Attributes: 6

- bread
- butter
- jam
- milk
- juice
- curd

=== Associator model (full training set) ===

Apriori

=====

Minimum support: 0.7 (3 instances)

Minimum metric <confidence>: 0.75

Number of cycles performed: 6

Generated sets of large itemsets:

Size of set of large itemsets L(1): 6

Size of set of large itemsets L(2): 9

Size of set of large itemsets L(3): 3

Best rules found:

1. curd=no 4 ==> milk=yes 4 <conf:(1)> lift:(1.25) lev:(0.16) [0] conv:(0.8)
2. milk=yes 4 ==> curd=no 4 <conf:(1)> lift:(1.25) lev:(0.16) [0] conv:(0.8)
3. butter=yes 3 ==> milk=yes 3 <conf:(1)> lift:(1.25) lev:(0.12) [0] conv:(0.6)
4. butter=yes 3 ==> curd=no 3 <conf:(1)> lift:(1.25) lev:(0.12) [0] conv:(0.6)
5. juice=yes 3 ==> jam=no 3 <conf:(1)> lift:(1.25) lev:(0.12) [0] conv:(0.6)
6. bread=yes curd=no 3 ==> milk=yes 3 <conf:(1)> lift:(1.25) lev:(0.12) [0] conv:(0.6)
7. bread=yes milk=yes 3 ==> curd=no 3 <conf:(1)> lift:(1.25) lev:(0.12) [0] conv:(0.6)
8. butter=yes curd=no 3 ==> milk=yes 3 <conf:(1)> lift:(1.25) lev:(0.12) [0] conv:(0.6)